LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 5. Conditional Statements

Lecture: 49. Chaining Comparision

Section 1: Lecture Summary

This lecture focused on two related but distinct Python concepts: **chaining comparison** and the **is / is not operators**. It began by explaining how multiple comparison conditions combined with "and" can be written more compactly as chained comparisons, a feature unique to Python. The instructor demonstrated with various examples how expressions like `a > b and b > c` can be written as `a > b > c` and how Python evaluates them. The lecture then shifted to the `is` operator, which compares object identity by checking if two variables reference the same object in memory, unlike equality `==`. Examples using numeric and string literals illustrated that Python reuses immutable literal objects, hence the same ID for separate variables with identical values. The related `is not` operator was also briefly introduced. Throughout, the concepts were illustrated with live Jupyter notebook examples.

Section 2: Key Concepts and Explanations

- **Chaining Comparison**

  - Traditional compound condition example:

do something

do something else

```
    if a > b and b > c:
    else:
```

  - Python allows chained comparisons for multiple conditions that share operands:

    - Syntax: `a > b > c`

    - Meaning: `(a > b) and (b > c)` with the middle variable (`b`) only written once.

    - Python evaluates the expressions from left to right, ensuring all conditions hold true.

- Conditions must be combined using logical **and** only—not **or**.

- All chained comparisons require the overlapping operand between adjacent comparison operators (e.g., `b` in `a > b > c`).

- This feature simplifies code readability and is unique to Python; it is not available in C, C++, or Java.

- **is and is not Operators**

- These operators compare **identity**, not equality.

- Identity means whether two variables point to the same object in memory.

- Every object in Python has a unique **ID** referring to its memory location.

- Use the built-in function `id()` to find an object's identity.

- Examples:

- Assigning `b = a` makes `b` a reference to the same object as `a`.

- For immutable literals like integers or strings, Python often uses the same object for identical values to save memory. For instance, `x = 25` and `y = 25` point to the same object, so `x is y` is True.

- The `is` operator returns `True` if two variables reference the same object.

- The complementary `is not` operator returns `True` if they reference different objects.

- This concept is especially important when manipulating objects where identity matters (e.g., checking if two variables are the same instance).

Section 3: Example Code and Use Cases

Chaining comparison example

Traditional compound condition

Chained comparison - equivalent and more concise

More chaining examples

Complex chaining with three comparisons

Using 'is' operator to check identity

Variables referring to the same object

Checking with id()

```
a = 10
b = 5
c = 3

print(a > b and b > c)  # Output: True

print(a > b > c)         # Output: True


a = 3
b = 5
c = 7

print(a < b < c)         # True (a < b and b < c)
print(a == b < c)        # False (a == b is False so whole is False)

d = 2
print(a > b < c > d)     # True if all parts hold


x = 25
y = 25

print(x is y)            # True (same integer literal, same object)

a = 'hello'
b = 'hello'

print(a is b)            # True (same string literal, same object)

a = 10
```

```
b = a
print(b is a)            # True


print(id(a))             # Same id value if 'a' and 'b' refer to the same
object
print(id(b))
```

- The above snippets demonstrate chaining for readability and the memory identity checks using `is`.

- Using `id()` confirms whether different variables share the same object in memory.

Section 4: Key Takeaways

- **Chaining comparison** syntax (`a > b > c`) is a Python-specific feature used to simplify compound logical comparisons connected by `and`.

- The chained comparison evaluates conditions left to right, sharing middle operands, equivalent to separate conditions combined by `and`.

- Chaining works only with `and`, not `or`.

- The `is` operator checks **object identity** (memory address equality), not value equality (`==`).

- Immutable Python literals (numbers, strings) are often stored once; variables with the same literal value may share the same object.

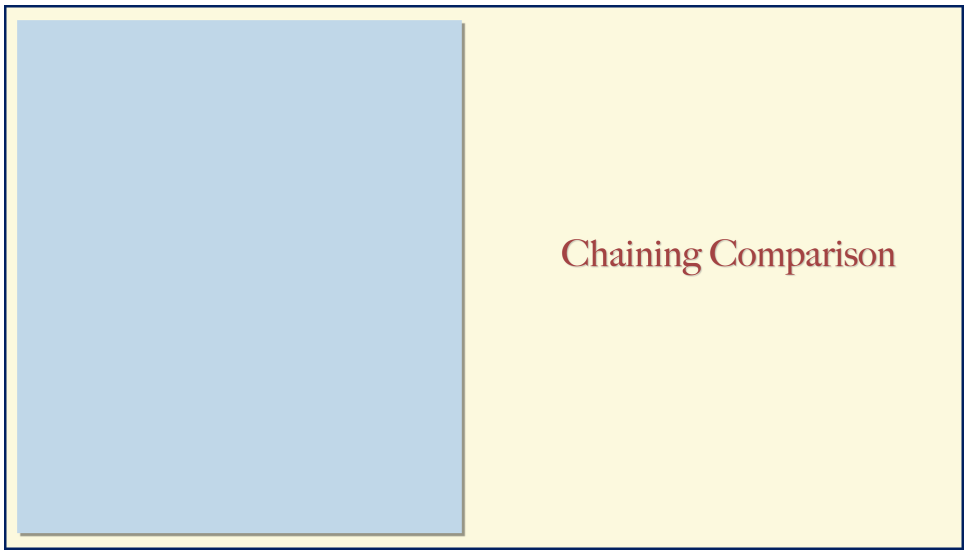
- Use `id()` to verify if two variables point to the same object.

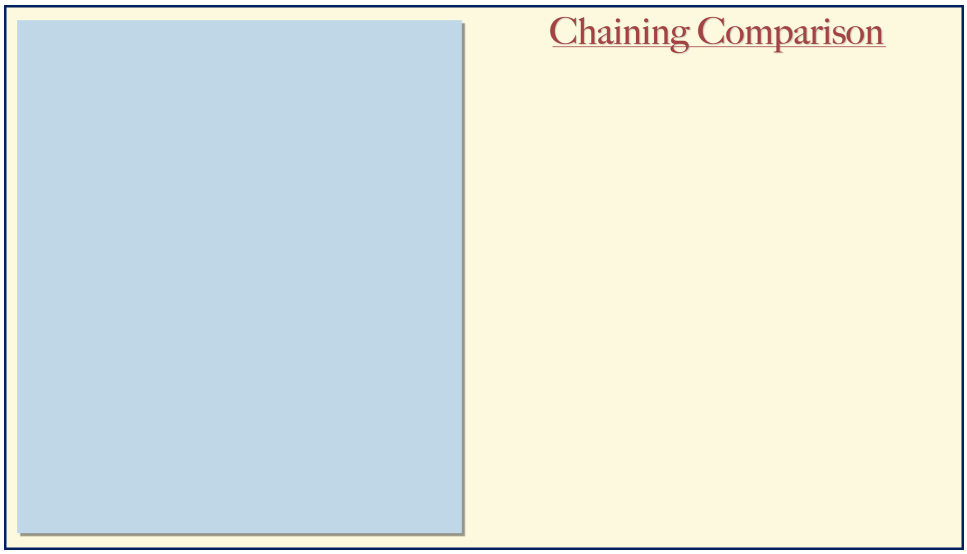
- `is not` is the negation of `is`.

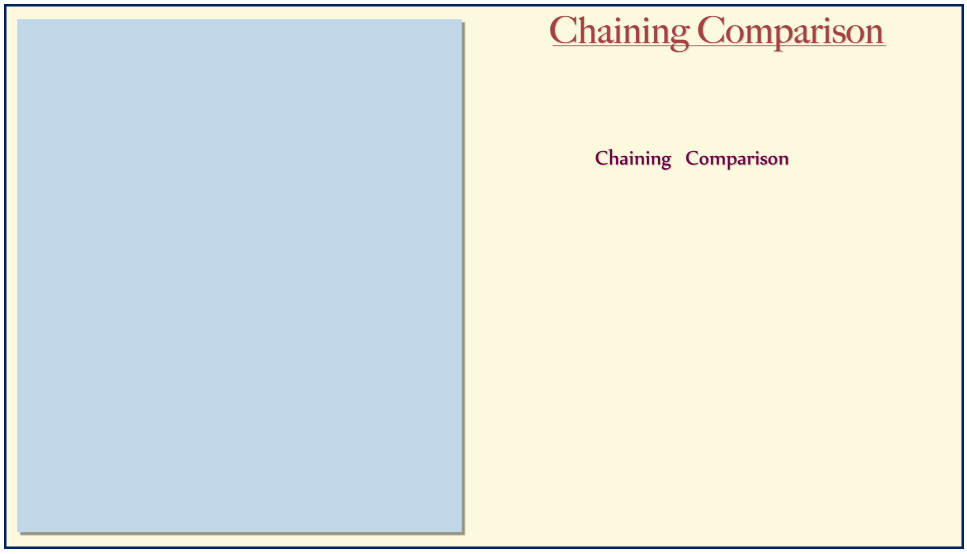
- Understanding identity vs equality is essential for certain Python programming scenarios, especially when dealing with mutable vs immutable objects.

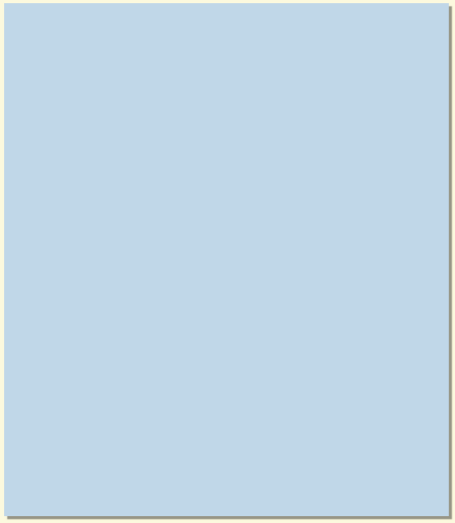
Lecture in Slides

Please Note: This document presents a curated selection of slides from the lecture; others have been excluded for conciseness.




Chaining Comparison

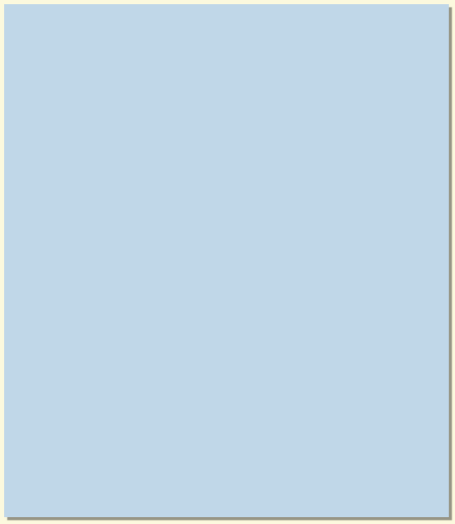

Chaining Comparison
Chaining Comparison



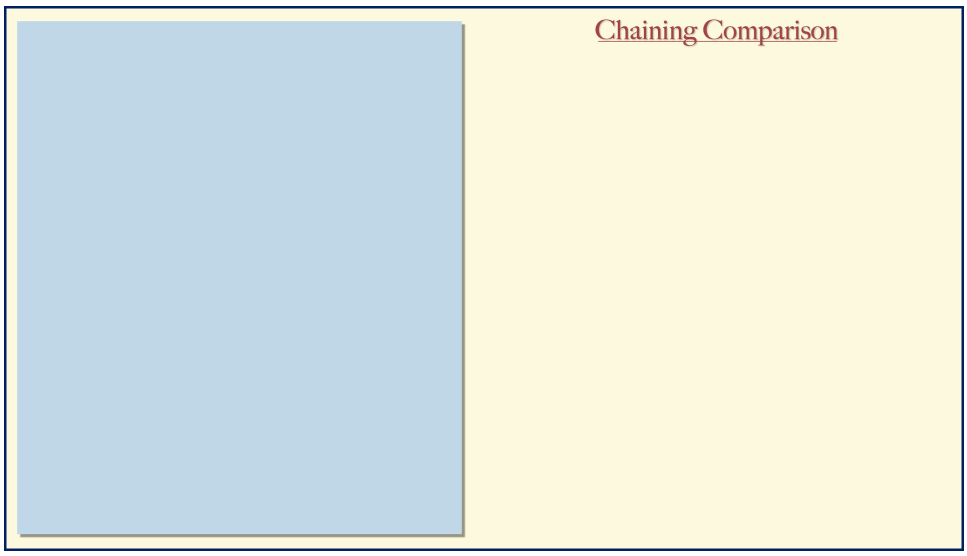
Chaining Comparison

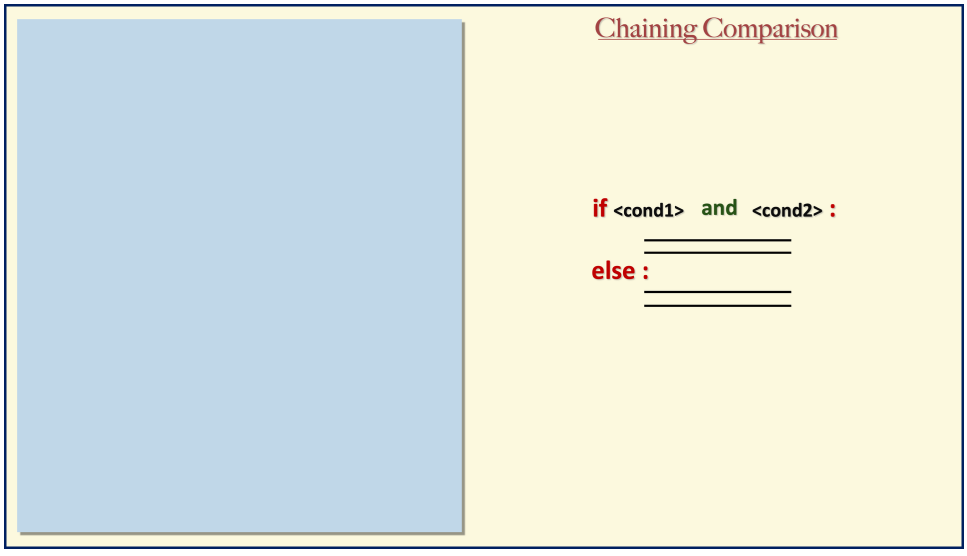
Chaining Comparison

is and is not



Chaining Comparison


Chaining Comparison


Chaining Comparison
if <cond1> and <cond2> :
else :

Chaining Comparison

a = 10 b = 5 c = 3

if <cond1> and <cond2> :

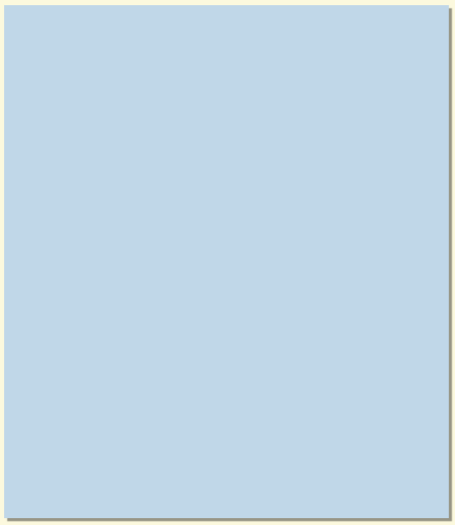
else :

Chaining Comparison

a = 10 b = 5 c = 3

if a > b and b > c :

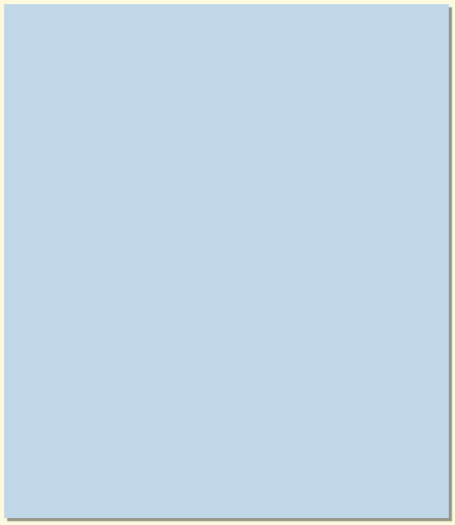
else :



Chaining Comparison

a = 10 b = 5 c = 3

```
if a > b and b > c :
```

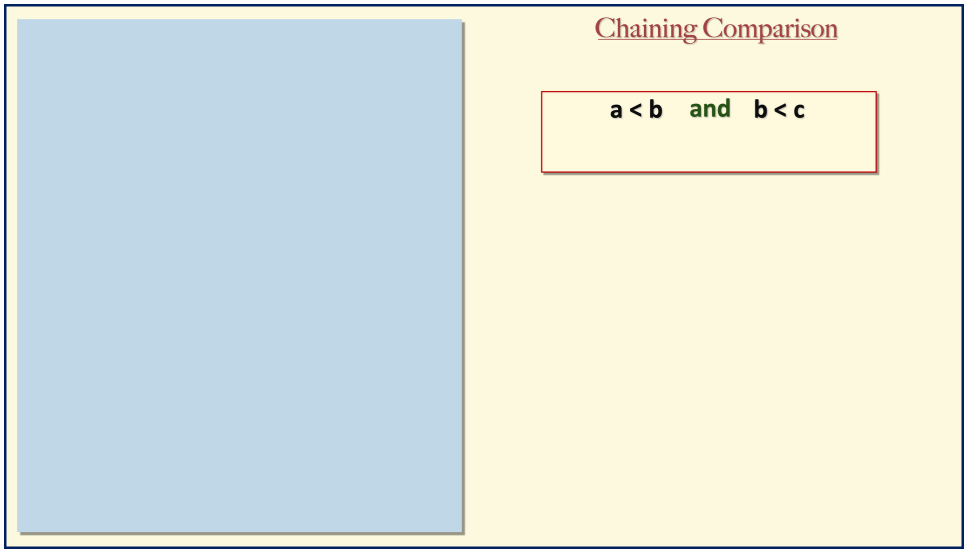


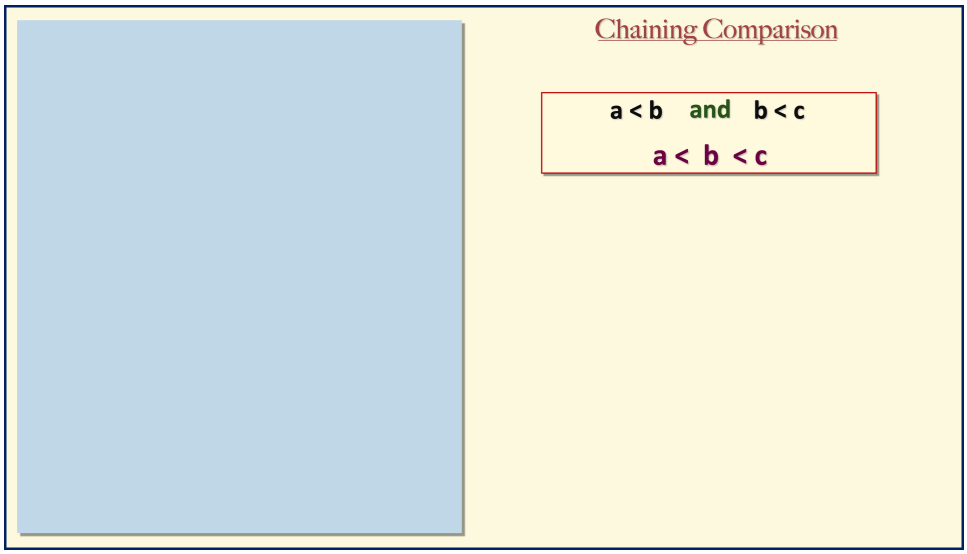
Chaining Comparison

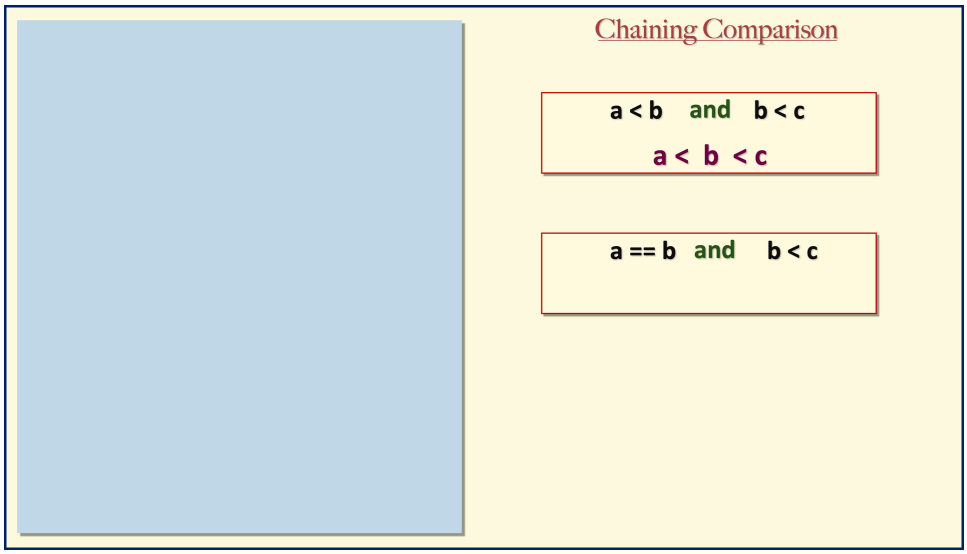
a = 10 b = 5 c = 3

```
if a > b and b > c :
```

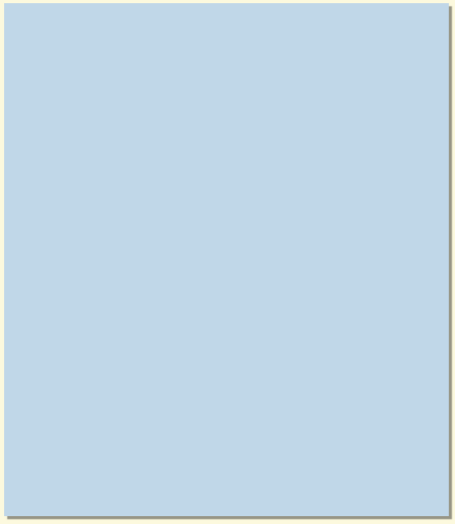
```
if a > b > c :
```


Chaining Comparison
a < b and b < c


Chaining Comparison
a < b and b < c
a < b < c


Chaining Comparison
a < b and b < c
a < b < c
a == b and b < c

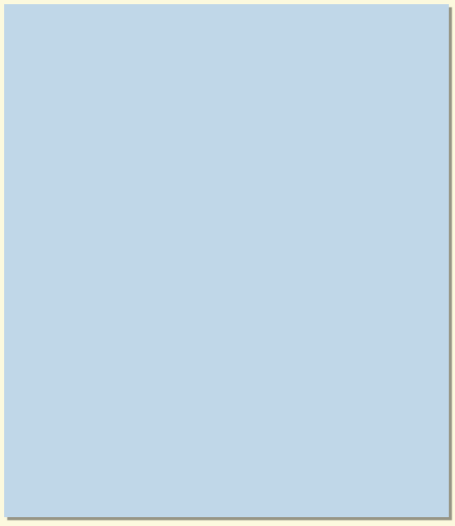
Chaining Comparison



```
a < b  and  b < c
a < b < c
```

```
a == b  and  b < c
a == b < c
```

Chaining Comparison



```
a < b  and  b < c
a < b < c
```

```
a == b  and  b < c
a == b < c
```

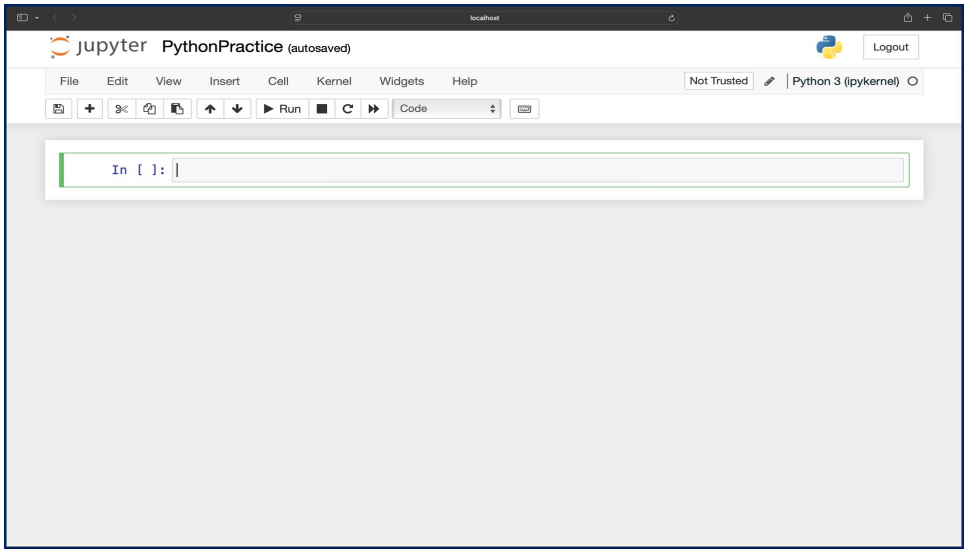
```
a > b  and  b < c  and  c > d
```

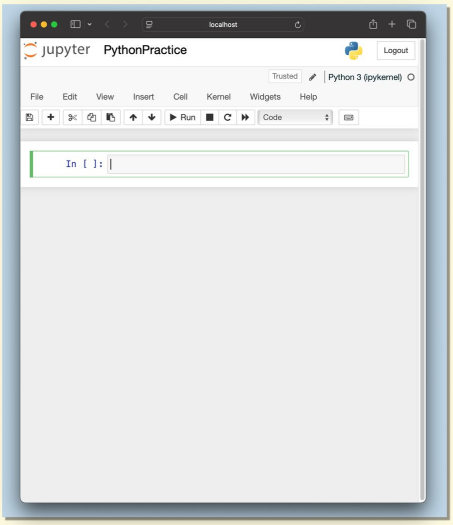
Chaining Comparison

```
a < b  and  b < c
a < b < c
```

```
a == b  and  b < c
a == b < c
```

```
a > b and b < c and c > d
a > b < c > d
```



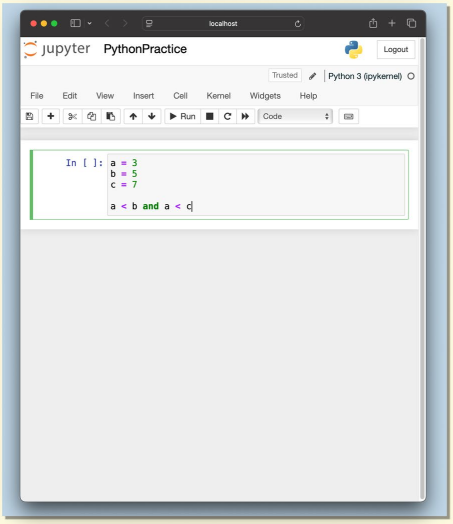


Chaining Comparison

```
a < b  and  b < c
a < b < c
```

```
a == b  and  b < c
a == b < c
```

```
a > b and b < c and c > d
a > b < c > d
```

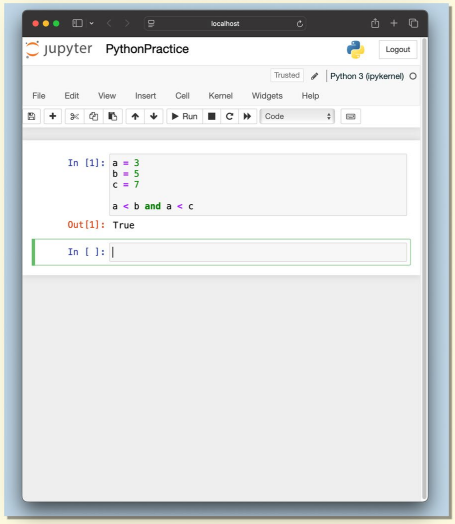


Chaining Comparison

```
a < b  and  b < c
a < b < c
```

```
a == b  and  b < c
a == b < c
```

```
a > b and b < c and c > d
a > b < c > d
```

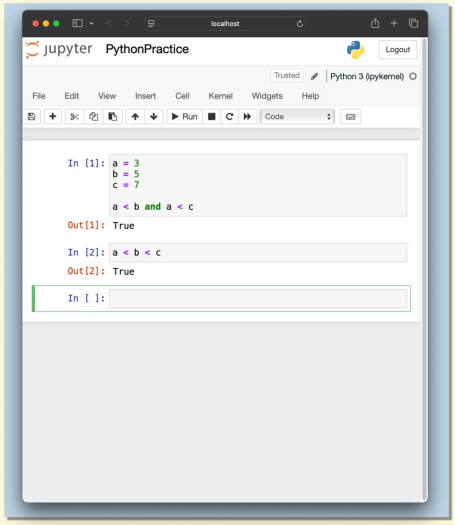


Chaining Comparison

```
a < b  and  b < c
a < b < c
```

```
a == b  and  b < c
a == b < c
```

```
a > b and b < c and c > d
a > b < c > d
```

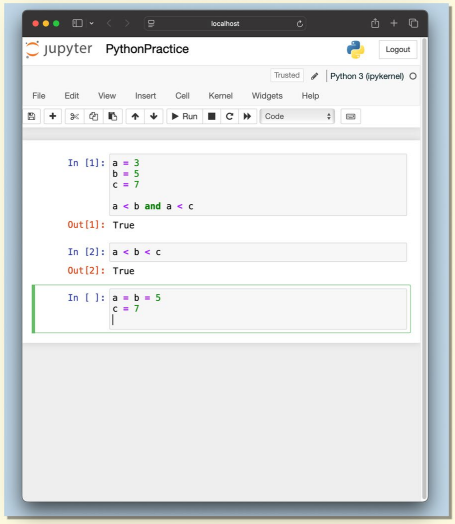


Chaining Comparison

```
a < b  and  b < c
a < b < c
```

```
a == b  and  b < c
a == b < c
```

```
a > b and b < c and c > d
a > b < c > d
```

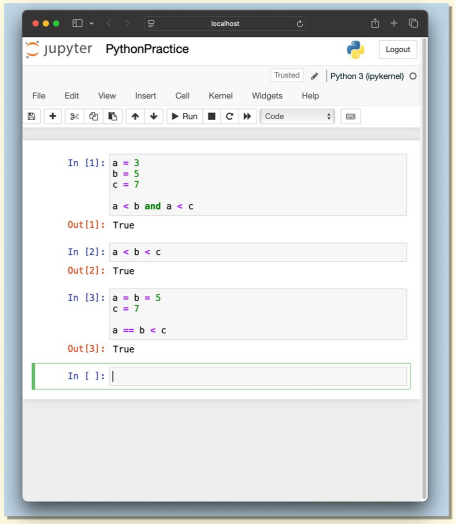


Chaining Comparison

```
a < b and b < c
a < b < c
```

```
a == b and b < c
a == b < c
```

```
a > b and b < c and c > d
a > b < c > d
```

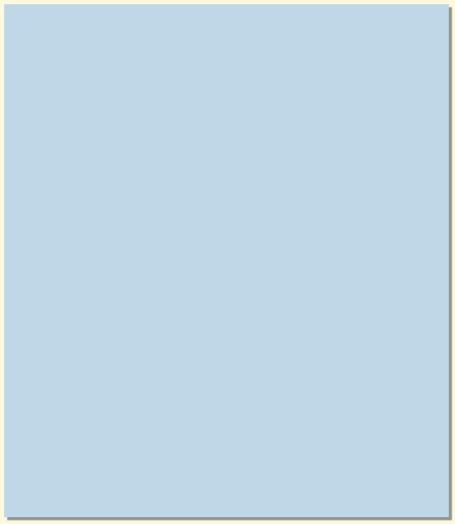


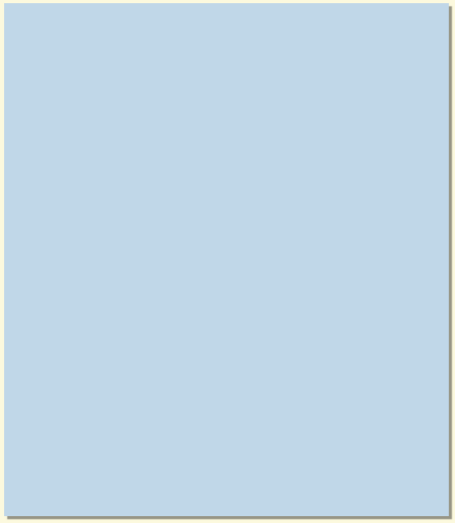
Chaining Comparison

```
a < b  and  b < c
a < b < c
```

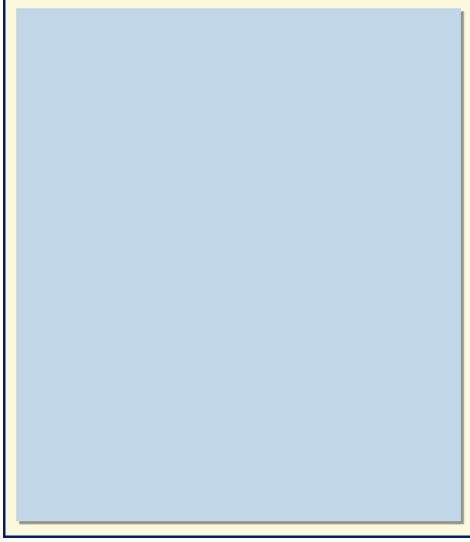
```
a == b  and  b < c
a == b < c
```

```
a > b and b < c and c > d
a > b < c > d
```





is and is not



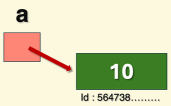
is and is not

is and is not

a = 10

is and is not

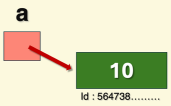
a = 10



is and is not

a = 10

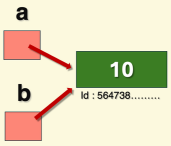
b = a



is and is not

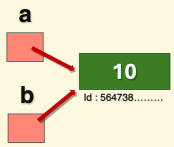
a = 10

b = a



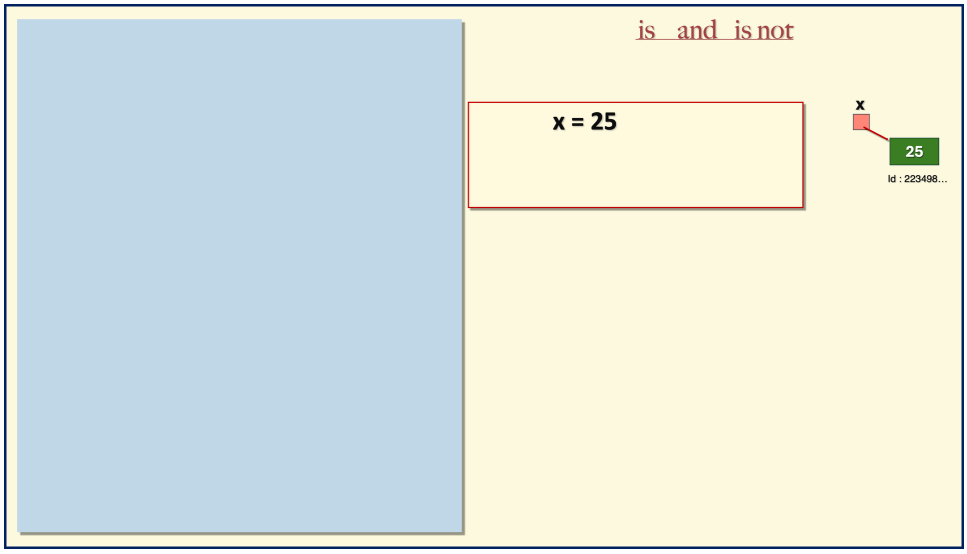
is and is not

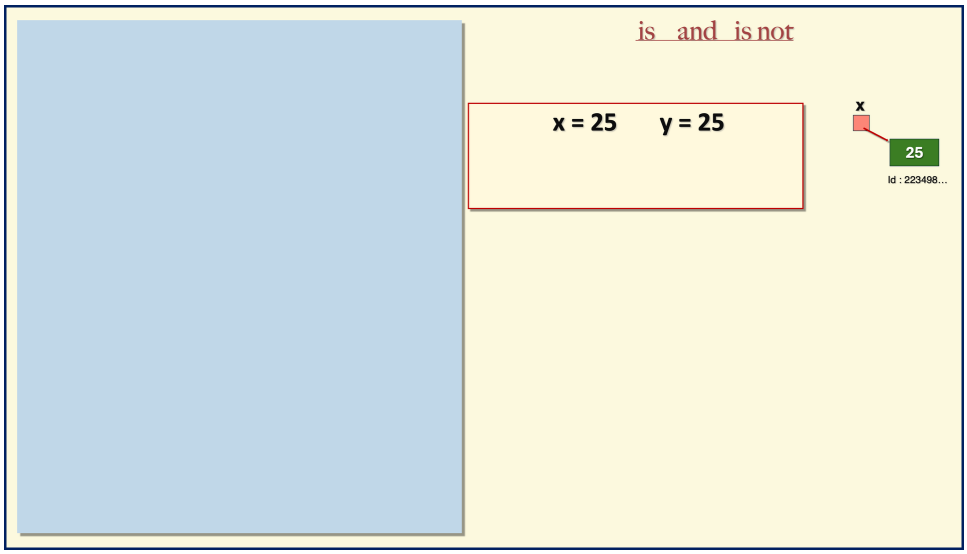
```
a = 10
b = a
if ( b is a ):
```

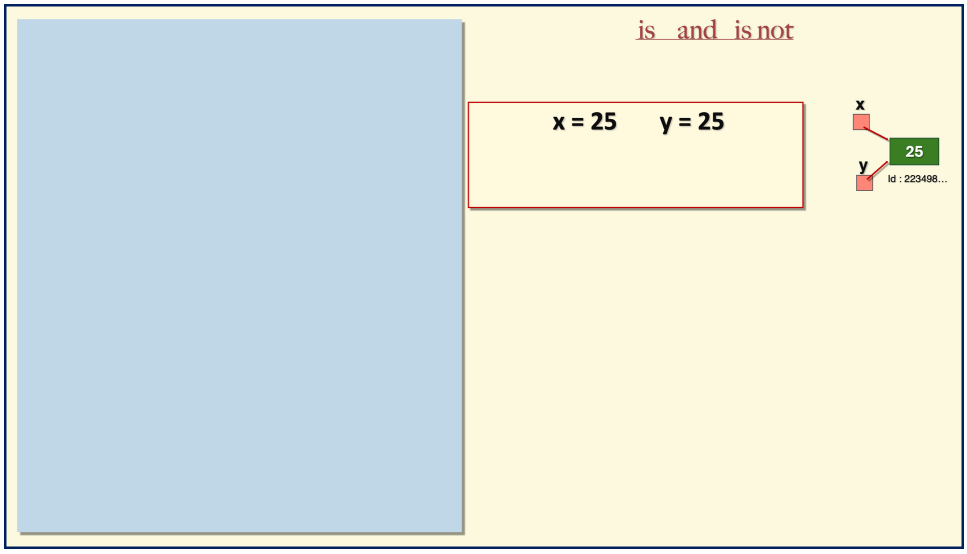


is and is not

```
x = 25
```

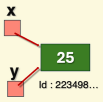

is and is not
x = 25
x
25
Id : 223498...


is and is not
x = 25 y = 25
x
25
Id : 223498...


is and is not
x = 25 y = 25
x
25
y
Id : 223498...

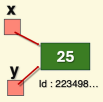
is and is not

```
x = 25      y = 25

if ( y  is  x ):
```



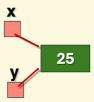
is and is not

```
x = 25      y = 25

if ( y  is  x ):  True
```



is and is not

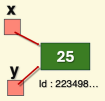
```
x = 25      y = 25

if ( y  is  x ):  True
```



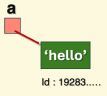

```
a = 'hello'
```

is and is not

```
x = 25      y = 25

if ( y  is  x ):  True
```



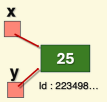

```
a = 'hello'
```



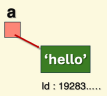
is and is not

x = 25 y = 25

if (y is x): True



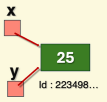
a = 'hello' b = 'hello'



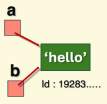
is and is not

x = 25 y = 25

if (y is x): True

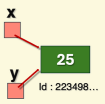


a = 'hello' b = 'hello'

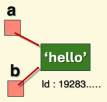


is and is not

```
x = 25      y = 25

if ( y  is  x ):   True
```

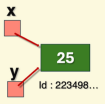



```
a = 'hello'      b = 'hello'

if ( a  is  b ):
```

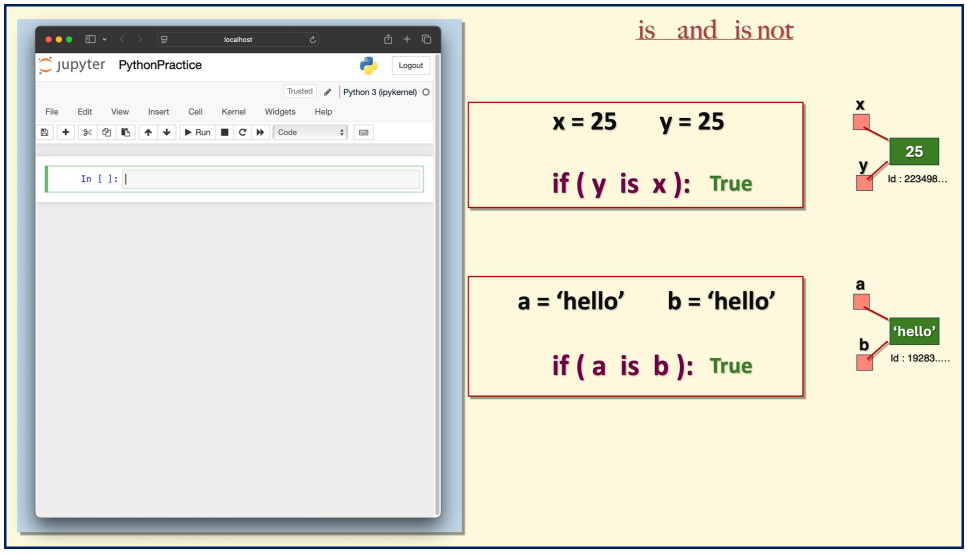


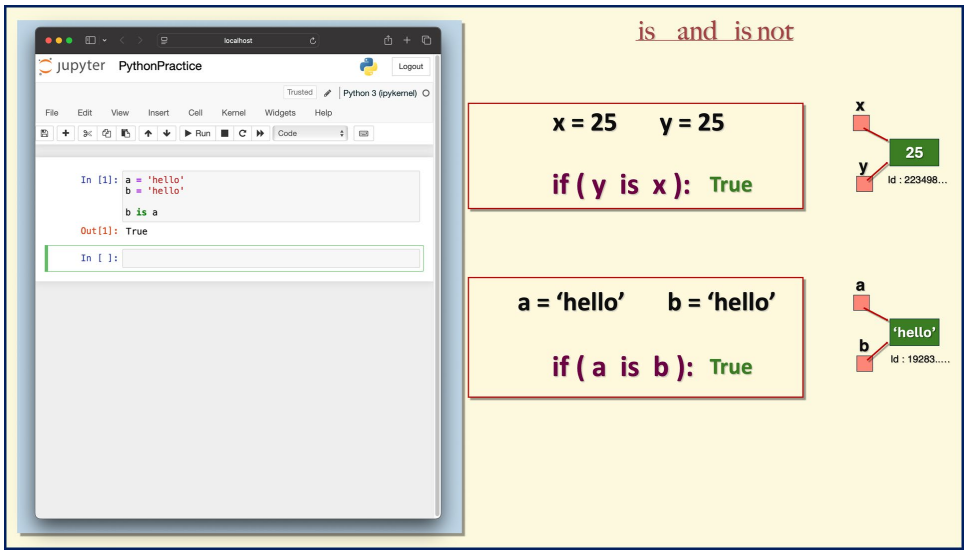
is and is not

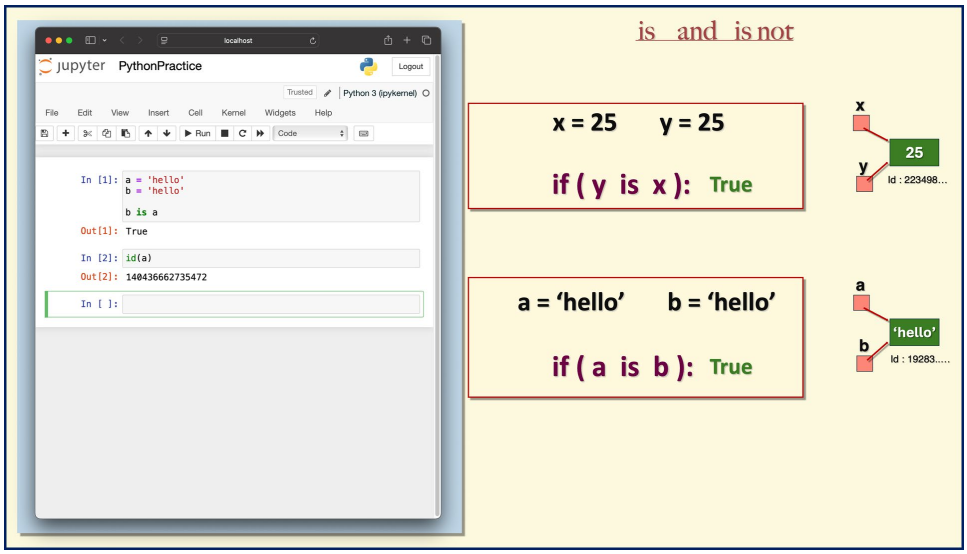
```
x = 25      y = 25

if ( y  is  x ):   True
```

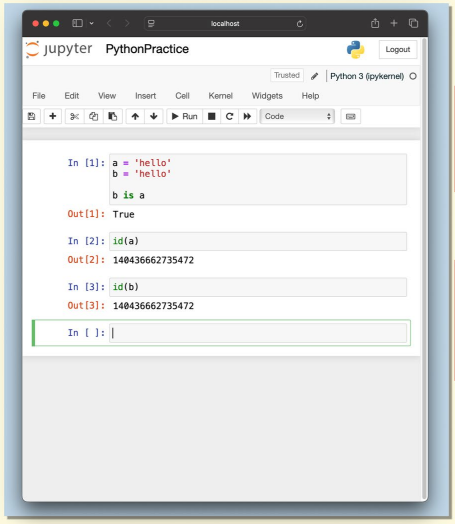



```
a = 'hello'      b = 'hello'

if ( a  is  b ):   True
```


localhost
jupyter PythonPractice
Logout
Trusted
Python 3 (ipykernel)
File Edit View Insert Cell Kernel Widgets Help
Run
Code
In [ ]:
is and is not
x = 25 y = 25
if (y is x): True
x
25
y
Id : 223498...
a = 'hello' b = 'hello'
if (a is b): True
a
'hello'
b
Id : 19283.....


is and is not
jupyter PythonPractice
Logout
Trusted
Python 3 (ipykernel)
File Edit View Insert Cell Kernel Widgets Help
Run
Code
In [1]: a = 'hello'
b = 'hello'
b is a
Out[1]: True
In [ ]:
x = 25 y = 25
if (y is x): True
x
y
25
Id : 223498...
a = 'hello' b = 'hello'
if (a is b): True
a
b
'hello'
Id : 19283.....

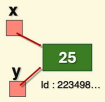

is and is not
jupyter PythonPractice
Logout
Trusted
Python 3 (ipykernel)
File Edit View Insert Cell Kernel Widgets Help
Run
Code
In [1]: a = 'hello'
b = 'hello'
b is a
Out[1]: True
In [2]: id(a)
Out[2]: 140436662735472
In [ ]:
x = 25 y = 25
if (y is x): True
x
y
25
Id : 223498...
a = 'hello' b = 'hello'
if (a is b): True
a
b
'hello'
Id : 19283.....



is and is not

x = 25 y = 25

if (y is x): True



a = 'hello' b = 'hello'

if (a is b): True